Lightweight LLMs for Network Attack Detection in IoT Networks

Piyumi Bhagya Sudasinghe*, Kushan Sudheera Kalupahana Liyanage*, Harsha S. Gardiyawasam Pussewalage†
*Department of Electrical and Information Engineering, University of Ruhuna, 81000 Matara, Sri Lanka
†Department of Information and Communication Technology, University of Agder (UiA), N-4898 Grimstad, Norway
Email: {piyumi.s, kushan}@eie.ruh.ac.lk; harsha.sandaruwan@uia.no

2025 Computing, Communications and IoT Applications (ComComAp) | 979-8-3315-9143-4/25/$31.00 ©2025 IEEE | DOI: 10.1109/ComComAp68359.2025.11353200

***Abstract*—The rapid growth of Internet of Things (IoT) devices has increased the scale and diversity of cyberattacks, exposing limitations in traditional intrusion detection systems. Classical machine learning (ML) models such as Random Forest and Support Vector Machine perform well on known attacks but require retraining to detect unseen or zero-day threats. This study investigates lightweight decoder-only Large Language Models (LLMs) for IoT attack detection by integrating structured-to-text conversion, Quantized Low-Rank Adaptation (QLoRA) fine-tuning, and Retrieval-Augmented Generation (RAG). Network traffic features are transformed into compact natural-language prompts, enabling efficient adaptation under constrained hardware. Experiments on the CICIoT2023 dataset show that a QLoRA-tuned LLaMA-1B model achieves an F1-score of 0.7124, comparable to the Random Forest (RF) baseline (0.7159) for known attacks. With RAG, the system attains 42.63% accuracy on unseen attack types without additional training, demonstrating practical zero-shot capability. These results highlight the potential of retrieval-enhanced lightweight LLMs as adaptable and resource-efficient solutions for next-generation IoT intrusion detection.**

***Index Terms*—Cybersecurity, IoT, LLMs, Network Attack Detection, QLoRA, RAG**

I. Introduction

The rapid proliferation of IoT and Industrial Control System (ICS) devices has substantially expanded the attack surface of modern networks [1]. As cyber threats grow in sophistication, robust network attack detection remains a critical component of cybersecurity infrastructure [2]. Traditional ML approaches, such as Random Forest (RF) and Support Vector Machine (SVM), achieve high accuracy on small-scale or binary intrusion detection tasks [3], [4], but their performance deteriorates in realistic multi-class IoT environments with heterogeneous attack types [5]. Moreover, these supervised models cannot effectively detect previously unseen (zero-day) attacks without retraining, limiting their adaptability to evolving threats.

Recent advances in Deep Learning (DL) and natural language processing (NLP) have introduced Large Language Models (LLMs) as versatile tools capable of modeling contextual dependencies across diverse data modalities [6]. Transformer-based LLMs have demonstrated strong potential for intrusion detection tasks by leveraging semantic understanding and sequence reasoning [6], [7]. Parameter-efficient fine-tuning methods such as Low-Rank Adaptation (LoRA) and Quantizes (QLoRA) further enable domain-specific adaptation with minimal computational and memory overhead, making them well-suited for IoT-scale deployments [8].

Transformer architectures are generally categorized as encoder-only, decoder-only, or encoder–decoder models. Encoder-based models (e.g., BERT, RoBERTa) are effective for classification and structured data representation, while decoder-only models (e.g., GPT-2, LLaMA, Mistral) are optimized for autoregressive text generation. Decoder models offer distinct advantages in reasoning, interpretability, and zero-shot adaptation, particularly when coupled with Retrieval-Augmented Generation (RAG) [9], [10].

In this work, we present a unified LLM-based framework that handles both known attacks via QLoRA fine-tuning and unknown attacks via RAG, overcoming the retraining limitations of traditional ML. Using the CICIoT2023 dataset [11], we conduct two experiments: (1) supervised fine-tuning of lightweight decoder-only LLMs for known attack detection, and (2) zero-shot detection of unseen attacks leveraging RAG. The main contributions of this work are:

- **LLM-based detection:** Adapting lightweight decoder LLMs with QLoRA for multi-class IoT attack detection (GPT-2, LLaMA-3.2-1B, Meta-LLaMA-3-8B, and Mistral-v0.3-7B).
- **Structured-to-text conversion:** Reformulating numerical features into concise natural language prompts.
- **Known and unknown attack generalization:** Leveraging RAG to enable zero-shot detection beyond traditional supervised models.
- **Comparative evaluation:** Benchmarking LLMs against classical ML classifiers using standard metrics.

The paper is organized as follows. Section II reviews intrusion detection from classical ML to modern LLMs. Section III details our methodology, including data preprocessing, QLoRA fine-tuning, and RAG. Section IV evaluates performance on known and unseen attacks. Finally, Section V concludes and outlines limitations and future work.

II. Related Work

This section surveys the evolution of network intrusion detection, from classical machine learning to contemporary LLM-based methods, culminating in the challenge of detecting previously unseen attacks.

Authorized licensed use limited to: Alliance University. Downloaded on October 14,2026 at 09:10:06 UTC from IEEE Xplore. Restrictions apply.

Classical machine learning techniques, such as Random Forest (RF) and Support Vector Machines (SVM), have been extensively employed for intrusion detection, demonstrating strong performance on balanced datasets [3], [4]. Nevertheless, their efficacy diminishes in multi-class scenarios with imbalanced distributions [5]. Deep learning approaches, particularly Convolutional Neural Networks (CNNs), have been explored to capture spatial and temporal patterns in network traffic, achieving improved detection accuracy [12]. However, these supervised methods generally cannot identify novel attack types without complete retraining.

The advent of Large Language Models (LLMs) has introduced advanced contextual reasoning capabilities in cybersecurity. Transformer-based architectures can effectively process structured network data transformed into textual prompts [6], [13]. Parameter-efficient fine-tuning methods, notably QLoRA [14], facilitate the adaptation of large models under computational constraints, enabling effective fine-tuning of decoder-only LLMs for known attack classification [15].

Detecting zero-day attacks remains a significant challenge. Meta-learning approaches allow models to adapt to new classes with minimal examples [16], while Generative Adversarial Networks (GANs) have been employed to synthesize data for rare attack types [17]. Few-shot and zero-shot learning strategies provide evaluation mechanisms for novel attacks [18], but typically require specialized architectures or extensive retraining. Recent deep-learning IDS frameworks have addressed zero-day and concept-drift scenarios via adaptive, multi-agent architectures [19]. In contrast, our methodology leverages LLMs with retrieval-augmented reasoning to capture semantic relationships across attack behaviors, enabling zero-shot generalization without modifying the model architecture or performing incremental updates.

Retrieval-Augmented Generation (RAG) has emerged as an effective mechanism to incorporate external knowledge, offering contextual evidence for tasks such as log anomaly detection without requiring parameter updates [20], [21]. Complementarily, hybrid architectures combining statistical detection with LLM-based reasoning have demonstrated improved detection performance in IoT environments [9], [10].

Building on these advances, our approach integrates QLoRA-based fine-tuning for known attacks and employs a RAG framework for previously unseen attacks, providing a unified system that supports both supervised classification and zero-shot generalization in complex IoT networks.

III. Methodology

This study employs an end-to-end methodology comprising dataset preparation, preprocessing, feature selection, structured-to-text conversion, and fine-tuning of lightweight decoder-only LLMs. The workflow integrates prompt design, tokenization, and parameter-efficient adaptation using QLoRA. Traditional machine learning models were implemented as baselines for performance comparison. Furthermore, a Retrieval-Augmented Generation (RAG) framework was incorporated to enable zero-shot detection of unseen attack types. The overall workflow of the proposed approach is illustrated in Fig. 1.

A. Data Preparation

The CICIoT2023 dataset [11] was used for this study, which comprises network traffic from 105 IoT devices and includes 34 classes (33 attacks and benign). To address the significant class imbalance, a balanced subset was created via random downsampling. From the dataset, 500 samples per class were allocated for model development (training and validation), while a separate hold-out set of 100 samples per class was reserved for final testing. The development set was further divided using an 80/20 train-validation split, a standard practice in IoT IDS research to manage class imbalance in CIC-based benchmarks. A summary of the data splits is provided in Table I.

For RAG experiments, 1,000 samples per class from 10 excluded attack types (unseen classes) were partitioned 70/30 for the retrieval knowledge base and testing, ensuring fair representation across training, retrieval, and evaluation.

B. Data Preprocessing and Feature Engineering

1) Feature Selection: The dataset contains 44 numerical features extracted from network flows. To reduce redundancy and multicollinearity, correlation-based feature selection was applied. Features with a Pearson coefficient above 0.98 were considered highly correlated; one feature from each correlated pair was removed manually after following the exploratory data analysis. The final selected 23 features are listed in Table II.

2) Data Normalization: Traditional ML models such as RF and SVM can be sensitive to differences in feature scales. To avoid this issue, Classical ML models were trained on normalized features using Scikit-learn's `StandardScaler`, standardizing each feature to zero mean and unit variance. For the LLM-based models, normalization was not applied since features were converted to natural language text. All feature values were represented as numerical values rounded to six decimal places (numerical text tokens).

C. Transformation for LLM Processing and RAG integration

To adapt the structured network traffic data for LLMs, several preprocessing steps were applied to ensure efficient tokenization and meaningful text representation. The primary objective was to transform tabular features into concise, semantically coherent natural language prompts suitable for fine-tuning.

1) Feature and Class Label Standardization: Feature and class names were standardized to maintain consistency and minimize unnecessary tokenization. Special characters such as hyphens and underscores were replaced with spaces, reducing input sequence length without affecting semantic clarity. Class labels were also simplified into meaningful and LLM-friendly names to ensure alignment with the model's vocabulary. Representative three examples from features and classes are shown in Table III, and the full set of selected seen/unseen classes is listed in Table IV.

Authorized licensed use limited to: Alliance University. Downloaded on October 14,2026 at 09:10:06 UTC from IEEE Xplore. Restrictions apply.

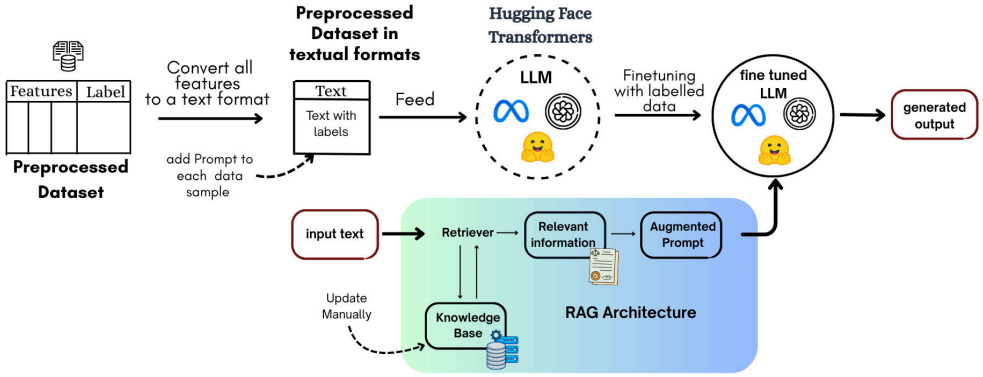


Fig. 1: Workflow summary chart

TABLE I: Data preparation split for both experiments

Dataset Split	Per-Class Count	Split Ratio	Classes	Usage
LLM: Training + Validation	500	80%	24	Training subset
		20%	24	Validation subset
LLM: Testing	100	–	24	Model evaluation
RAG Knowledge Base + RAG Test	1,000	70%	10	RAG Knowledge Base (unseen)
		30%	10	RAG Test (unseen)

TABLE II: Feature Descriptions

Feature Name	Description
Header_Length	Packet header length
Protocol Type	Transport/network layer protocol
Time_To_Live	IP packet TTL value
psh_flag_number	Packets with PSH flag set
ack_flag_number	Packets with ACK flag set
ack_count	Acknowledged packets in flow
syn_count	Packets with SYN flag set
fin_count	Packets with FIN flag set
rst_count	Packets with RST flag set
HTTP	HTTP usage in flow (binary)
HTTPS	HTTPS usage in flow (binary)
DNS	DNS usage in flow (binary)
TCP	TCP transport protocol (binary)
UDP	UDP transport protocol (binary)
ICMP	ICMP network protocol (binary)
Tot sum	Total packet length in flow
Min	Min. packet size in flow
Max	Max. packet size in flow
AVG	Avg. packet size in flow
Std	Std. dev. of packet sizes in flow
IAT	Inter-arrival time of packets
Number	Total packets in flow
Rate	Packet transmission rate in flow
Label	Attack/benign class label

TABLE III: Feature and Class Label Standardization Examples

Feature Name Examples:
Header_Length → Header Length
rst_count → Packets with RST Flag
IAT → Time Between Packets
Class Label Examples:
DDOSUDP_FLOOD → ddos udp flood
MITMARPSPOOFING → mitm arp spoofing
DDOSSYN_FLOOD → ddos synchronize flood

TABLE IV: Seen and Unseen Attack Classes Used in the Study

Seen Classes (Training/Validation)	
benign	ddos ack fragmentation
ddos icmp flood	ddos icmp fragmentation
ddos pshack flood	ddos rst fin flood
ddos synchronize flood	ddos synonymousip flood
ddos tcp flood	ddos udp flood
ddos udp fragmentation	dns spoofing
dos http flood	dos synchronize flood
dos tcp flood	dos udp flood
mirai greeth flood	mirai greip flood
mirai udp plain	mitm arp spoofing
recon host discovery	recon os scan
recon port scan	vulnerability scan
Unseen Classes (RAG Evaluation)	
backdoor malware	browser hijacking
command injection	ddos http flood
ddos slow loris	dictionary brute force
recon ping sweep	sql injection
cross site scripting	uploading attack

2) Prompt Engineering and Tokenization: Each record was transformed into a structured prompt containing the task description, possible attack classes and feature–value pairs. An example is shown in Table V. During training, the ground-truth label followed the `Answer:` token and during the inference, the model predict the class after this token.

Tokenization was performed using Hugging Face's `AutoTokenizer` with truncation to 512 tokens. Since decoder-only models lack a default padding token, the end-of-sequence (`<eos>`) token was used for padding. A custom data collator extended from `DataCollatorForLanguageModeling` ensured that only label tokens (after `Answer:`) contributed to loss computation, focusing optimization solely on predicting the correct class.

Authorized licensed use limited to: Alliance University. Downloaded on October 14,2026 at 09:10:06 UTC from IEEE Xplore. Restrictions apply.

TABLE V: Example Prompt with generated output in bold text

Task: Network Attack Classification *Input Features: {Header Length = 20.0; Protocol Type = 6.0; IP Time to Live = 64.0; Flow Packet Transmission Rate = 41913.7; . . . }* *Possible Classes: [benign, ddos icmp flood, browser hijacking, backdoor, . . .]* *Answer:* ***benign***

3) Parameter-Efficient Fine-Tuning with QLoRA: For the LLM fine tuning, the QLORA technique was used. QLoRA combines 4-bit quantization with Low-Rank Adaptation to fine-tune large decoders efficiently under resource constraints. Model weights are loaded in 4-bit precision with mixed FP16 computation, double quantization, and NF4 for numerical stability. LoRA adapters are applied to attention projections with rank $r = 16$, scaling factor $\alpha = 32$, dropout 0.1, and frozen biases. Most parameters remain frozen, so only a small subset is trainable, reducing memory and computational requirements while maintaining performance across all LLMs.

4) Generalization to Unseen Attack Types with RAG: To evaluate model generalization, a Retrieval-Augmented Generation (RAG) framework was integrated into the fine-tuned LLM workflow. Attack types excluded from training were used as unseen classes. Their feature vectors were embedded to construct a retrieval knowledge base. During inference, each test instance was compared against this base using cosine similarity, and the top 20 most similar samples were identified. The top 3 retrieved examples were concatenated with the query instance (within token constraints) to form the final prompt.

This prompt provided contextual reference for the LLM to infer unseen attack types. The model's predictions were generated based on both the query flow and the retrieved exemplars. Evaluation metrics included Accuracy, Precision, Recall, and F1-score for classification, and top-k (k=3) recall for retrieval effectiveness. This methodological setup enabled structured assessment of zero-shot generalization capability using retrieval support.

5) Hardware Setup: LLM experiments were conducted on two separate GPUs: NVIDIA RTX 4080 (16 GiB CUDA memory) for GPT-2 and LLaMA-3.2-1B, and NVIDIA RTX 4090 (32 GiB CUDA memory) for Mistral-7B and LLaMA-3-8B fine-tuning.

IV. Results

This section presents the outcomes of the two experiments: (1) direct classification using fine-tuned models and classical baselines, and (2) RAG-based evaluation for unseen attacks.

A. Experiment 1: Direct Classification

1) Model Training and Hyperparameters: In the first phase, the models are trained and evaluated on attack classes included in the training set. Classical ML classifiers (RF, SVM, and Logistic Regression (LR)) are trained directly on normalized feature vectors. In parallel, decoder-based LLMs including GPT-2, LLaMA-3.2-1B, Meta-LLaMA-3-8B, and Mistral-v0.3-7B are fine-tuned on textual prompts constructed from the same features. All LLM models were fine-tuned using the QLoRA setup described earlier. The main fine-tuning settings are summarized in Table VI.

2) Parameter Efficiency: Table VII reports the number and proportion of trainable parameters. Only a small fraction of each model's parameters are updated, confirming QLoRA's parameter efficiency and suitability for resource-limited deployment.

3) Performance Evaluation: Model performance was evaluated using Accuracy, Precision, Recall, and F1-score on the test dataset. Results are summarized in Table VIII.

Among baseline models, *Random Forest* achieved the highest performance (Accuracy = 0.7171, F1 = 0.7159). For fine-tuned LLMs, LLaMA 3.2-1B achieved the best F1-score (**0.7124**) with balanced metrics, followed by Mistral-7B (F1 = 0.6992). Although LLMs require higher inference time, their runtime remains practical for offline or batch intrusion analysis. LLaMA-3.2-1B offers a favorable balance between accuracy and compute cost (235 s), showing that lightweight LLMs can deliver strong detection under modest hardware while enabling zero-shot generalization not achievable with traditional models.

4) Per-Class Performance: LLaMA 3.2-1B, the best-performing fine-tuned model, was used for detailed class-wise evaluation. Results are presented in Table IX.

High F1-scores (≥ 0.98) for *DDoS* and *Mirai*-based attacks such as *ddos icmp flood*, *mirai udp plain*, and *ddos pshack flood* indicate strong detection of highly distinctive attack signatures. Conversely, lower F1-scores for *recon os scan*, *vulnerability scan*, and *ddos synchronize flood* reveal difficulty in differentiating subtle or overlapping network behaviors.

The per-class metric plot (Fig. 2) visually reinforces the quantitative findings, highlighting strong predictive accuracy for most attack types while revealing misclassifications in classes that share similar traffic characteristics.

TABLE VI: LLM Fine-tuning Configuration

Parameter	**Value**
LoRA Settings	Rank $r = 16$, Scaling $\alpha = 32$, Dropout = 0.1
Target Layers	Attention/ MLP
Epochs	3–5
Batch Size	4 (7B and 8B model),16 (other models)
Learning Rate	varies (0.0005 or 0.00005)
Optimizer	AdamW
Weight Decay	0.2
Mixed Precision	FP16

TABLE VII: Trainable Parameters Compared to Total Model Size

Model	**Trainable**	**Total**	**Trainable %**
GPT-2	1.57M	356.40M	0.44%
LLaMA-3.2 1B	2.36M	1,238.17M	0.19%
Mistral-v0.3-7B	9.44M	7,257.46M	0.13%
LLaMA-3.1 8B	9.44M	8,039.70M	0.12%

Authorized licensed use limited to: Alliance University. Downloaded on October 14,2026 at 09:10:06 UTC from IEEE Xplore. Restrictions apply.

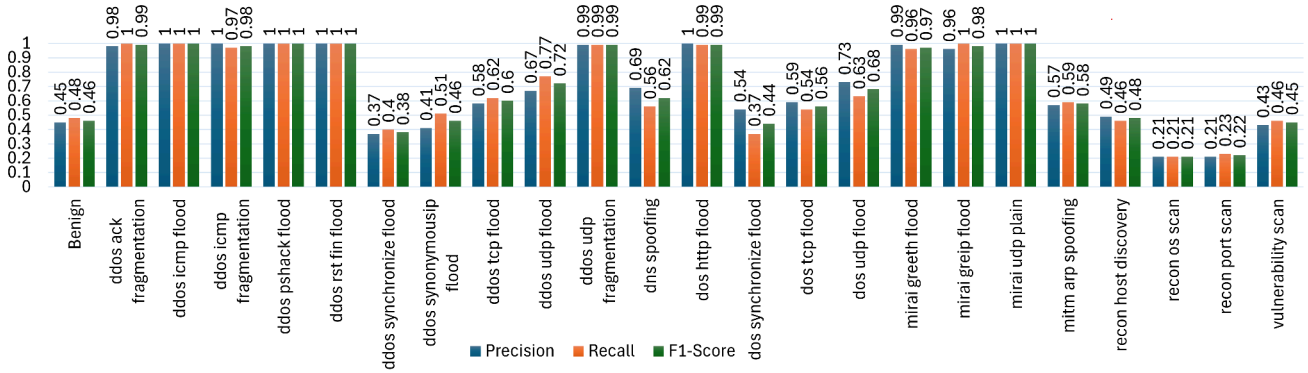


Fig. 2: Per-Class Precision, Recall, and for LaMA 3.2-1B

TABLE VIII: Test Set Evaluation Metrics for All Models

Model	Accuracy	F1	Precision	Recall	Runtime/s
LR	0.6792	0.6778	0.6792	0.6676	0.0004
RF	**0.7171**	**0.7159**	0.7171	0.7143	0.0245
SVM	0.6470	0.6761	0.6650	0.6650	0.6718
LLaMA-3.2-1B	0.7117	0.7124	0.7173	0.7117	235.16
Mistral-7B	0.7104	0.6992	0.7210	0.7104	1050.58
LLaMA-3.8-8B	0.6796	0.6792	0.6824	0.6796	679.51
GPT-2	0.6567	0.6271	0.6617	0.6567	191.05

TABLE IX: Per-Class Classification Report for Test Set (LLaMA 3.2-1B)

Class	Precision	Recall	F1	Support
Benign	0.45	0.48	0.46	100
ddos ack fragmentation	0.98	1.00	0.99	100
ddos icmp flood	1.00	1.00	1.00	100
ddos icmp fragmentation	1.00	0.97	0.98	100
ddos pshack flood	1.00	1.00	1.00	100
ddos rst fin flood	1.00	1.00	1.00	100
ddos synchronize flood	0.37	0.40	0.38	100
ddos synonymousip flood	0.41	0.51	0.46	100
ddos tcp flood	0.58	0.62	0.60	100
ddos udp flood	0.67	0.77	0.72	100
ddos udp fragmentation	0.99	0.99	0.99	100
dns spoofing	0.69	0.56	0.62	100
dos http flood	1.00	0.99	0.99	100
dos synchronize flood	0.54	0.37	0.44	100
dos tcp flood	0.59	0.54	0.56	100
dos udp flood	0.73	0.63	0.68	100
mirai greeth flood	0.99	0.96	0.97	100
mirai greip flood	0.96	1.00	0.98	100
mirai udp plain	1.00	1.00	1.00	100
mitm arp spoofing	0.57	0.59	0.58	100
recon host discovery	0.49	0.46	0.48	100
recon os scan	0.21	0.21	0.21	100
recon port scan	0.21	0.23	0.22	100
vulnerability scan	0.43	0.46	0.45	100
Macro Avg	0.70	0.70	0.70	2400
Weighted Avg	0.70	0.70	0.70	2400
Accuracy		0.70		2400

B. Experiment 2: RAG-Based Evaluation for Unseen Attacks

1) Numerical Embedding Retrieval: Before passing input prompts to the LLM, we evaluated the ability of the numerical embeddings to retrieve relevant examples from the RAG knowledge base. For each test instance, we computed cosine similarity against all examples in the knowledge base and considered the top 20 candidates. The metric used is Top-3 Recall, which indicates whether at least one of the top 3 retrieved candidates matches the actual class of the test sample. Per-Class evaluation is shown in Table XI.

Across all 3,000 test samples, the overall Top-3 Recall was 63.27% (1898/3000), indicating that for nearly two-thirds of the test instances, at least one of the top 3 candidates retrieved from the knowledge base matched the true class. This retrieval step ensures that the subsequent LLM input is enriched with relevant examples, improving its ability to classify unseen or difficult attack types.

2) RAG Prompt Construction and Generation Settings: From the top 20 retrieved candidates, the three most similar were concatenated with the current input. Prompts were truncated to 1015 tokens and limited to six generated tokens. The final prediction was extracted after the last `Answer:` token. The Table X illustrates a RAG-augmented prompt example used and its model output (shown in bold).

TABLE X: RAG-augmented prompt example with generated output in bold text

Retrieved Examples:
Input: {Header Length=27.6; Protocol Type=6.0; IP TTL=57.3; ...; Total Packets=10}
Answer: recon ping sweep
Input: {Header Length=25.6; Protocol Type=6.0; IP TTL=78.8; ...; Total Packets=10}
Answer: sql injection
Input: {Header Length=21.6; Protocol Type=6.0; IP TTL=56.8; ...; Total Packets=10}
Answer: recon ping sweep
Task: Network Attack Classification;
Possible Classes: [recon ping sweep, sql injection]
Example Input: {Header Length=25.6; Protocol Type=6.0; IP TTL=52.8; ...; Total Packets=10}
Answer: **recon ping sweep**

This format helps the model reason about unseen attacks by grounding predictions in semantically similar examples.

Authorized licensed use limited to: Alliance University. Downloaded on October 14,2026 at 09:10:06 UTC from IEEE Xplore. Restrictions apply.

3) RAG-Based Classification Performance: Per-class evaluation metrics for the RAG-based experiment are presented in Table XII. The fine-tuned model achieved an overall accuracy of 42.63% on previously unseen attack classes, demonstrating its capacity for zero-shot generalization without any additional supervised retraining.

TABLE XI: Top-3 Recall (%) per Class Using Numerical Embeddings

Attack Class	Top-3 Recall (%)
Recon Ping Sweep	59.59 (174/292)
DDoS Slow Loris	99.06 (317/320)
Browser Hijacking	59.09 (182/308)
Backdoor Malware	41.19 (131/318)
Dictionary Brute Force	52.76 (153/290)
Command Injection	60.42 (174/288)
SQL Injection	59.25 (173/292)
DDoS HTTP Flood	97.36 (295/303)
Cross Site Scripting	51.70 (137/265)
Uploading Attack	50.00 (162/324)

TABLE XII: Per-Class Results for RAG-Based Evaluation

Class	Precision	Recall	F1	Support
Recon Ping Sweep	0.35	0.35	0.35	292
DDoS Slow Loris	0.96	0.94	0.95	320
Browser Hijacking	0.39	0.35	0.37	308
Backdoor Malware	0.24	0.21	0.22	318
Dictionary Brute Force	0.29	0.31	0.30	290
Command Injection	0.33	0.38	0.35	288
SQL Injection	0.30	0.28	0.29	292
DDoS HTTP Flood	0.94	0.96	0.95	303
Cross Site Scripting	0.16	0.20	0.18	265
Uploading Attack	0.26	0.24	0.25	324
Macro Avg	0.38	0.38	0.38	2990
Weighted Avg	0.43	0.43	0.43	2990
Accuracy		0.426		2990

The results in Table XII indicate that RAG-enhanced inference substantially improves the LLM's ability to recognize semantically distinct attack types. High precision and recall scores for classes such as *DDoS Slow Loris* and *DDoS HTTP Flood* suggest that the retrieval mechanism effectively supplements contextual understanding for well-represented behavioral patterns in the external knowledge base. In contrast, lower performance on complex or less distinctive attacks like *Backdoor Malware*, *Cross Site Scripting*, and *Uploading Attack* reflects the inherent challenge of modeling subtle variations in traffic features when semantic overlap exists between categories.

These findings demonstrate that integrating retrieval-based augmentation helps bridge the gap between seen and unseen classes, enabling LLMs to extend beyond fixed supervised boundaries typical of classical ML models. Although overall accuracy remains moderate, the RAG framework shows promise for incremental adaptation, where the knowledge base can be continuously enriched with new attack exemplars to improve coverage and robustness against emerging IoT threats. This suggests a viable direction for future work toward hybrid retrieval–generation intrusion detection frameworks.

V. Conclusions and Future Work

This study shows that lightweight decoder-only LLMs, adapted using structured-to-text conversion, QLoRA fine-tuning, and RAG, form an efficient unified framework for IoT intrusion detection. On CICIoT2023, the fine-tuned LLaMA-1B model achieved an F1-score of 0.7124, matching the Random Forest baseline (0.7159) on known attacks, while the RAG-enhanced model reached 42.63% accuracy on unseen attack types without additional training, demonstrating practical zero-shot capability beyond traditional ML. By combining parameter efficiency with retrieval-based context grounding, the proposed approach offers a scalable and resource-aware solution suitable for evolving IoT environments. A key limitation is that results were obtained on a single dataset, and broader validation across heterogeneous IoT benchmarks is required. Future work will explore improved retrieval strategies, larger and more diverse knowledge bases, and lightweight ensemble methods to enhance detection of subtle or overlapping attack patterns.

References

[1] M. Zong *et al.*, "Integrating large language models with internet of things," *Discover Internet of Things*, 2025, doi:10.1007/s43926-024-00083-4.

[2] R. K. Panchal *et al.*, "A survey on network-based intrusion detection system using learning techniques," in *Proc. ICIPCN*, 2024, doi:10.1109/ICIPCN63822.2024.00128.

[3] Y. Chang *et al.*, "Network intrusion detection based on random forest and svm," in *IEEE CSE/EUC*, 2017, doi:10.1109/CSE-EUC.2017.118.

[4] S. Hiremath *et al.*, "Machine learning models for intrusion detection system," in *IEEE CSNT*, 2025, doi:10.1109/CSNT64827.2025.10967597.

[5] R. A. Disha *et al.*, "Performance analysis of machine learning models for intrusion detection system," *Cybersecurity*, 2022, doi:10.1186/s42400-021-00103-8.

[6] R. Kaur *et al.*, "Harnessing the power of language models in cybersecurity," *Int. J. Inf. Manage. Data Insights*, 2025, doi:10.1016/j.jjimei.2024.100315.

[7] X. Zhang *et al.*, "Large language models powered malicious traffic detection," *IEEE Network*, 2025, doi:10.1109/MNET.2025.3583088.

[8] Y. Mao *et al.*, "A survey on lora of large language models," *Front. Comput. Sci.*, 2025, doi:10.1007/s11704-024-40663-9.

[9] M. F. Al-Hammouri *et al.*, "Hybrid llm-enhanced intrusion detection for zero-day threats in iot," *arXiv:2507.07413*, 2025.

[10] F. Y. Loumachi *et al.*, "Advancing cyber incident timeline analysis through rag and llms," *Computers*, 2025.

[11] A. Hammal *et al.*, "Ciciot2023: A real-time dataset for large-scale attacks in iot," *Sensors*, 2023, doi:10.3390/s23135941.

[12] M. A. Ferrag *et al.*, "Deep learning for cyber security intrusion detection," *J. Inf. Secur. Appl.*, 2020, doi:10.1016/j.jisa.2019.102419.

[13] H. Kheddar, "Transformers and llms for efficient intrusion detection systems," *Inf. Fusion*, 2025, doi:10.1016/j.inffus.2025.103347.

[14] T. Dettmers *et al.*, "Qlora: Efficient finetuning of quantized llms," in *NeurIPS*, 2023, doi:10.1109/BDCAT63179.2024.00021.

[15] P. R. B. Houssel *et al.*, "Towards explainable network intrusion detection using llms," in *IEEE/ACM BDCAT*, 2024.

[16] F. S. Alrayes *et al.*, "An adaptive framework for intrusion detection in iot using maml," *Sensors*, 2025, doi:10.3390/s23135941.

[17] S. A. R. Shirazi *et al.*, "Android malware intrusion detection using zero-shot learning gans," *Sir Syed Univ. Res. J.*, 2023.

[18] T. Althiyabi *et al.*, "Enhancing iot security: A few-shot learning approach for intrusion detection," *Mathematics*, 2024, doi:10.3390/math12071055.

[19] R. M. Zaki *et al.*, "Hybrid classifier for detecting zero-day attacks on iot networks," *Mesopotamian Journal of CyberSecurity*, 2024.

[20] J. Pan *et al.*, "Raglog: Log anomaly detection using retrieval-augmented generation," *arXiv:2311.05261*, 2023.

[21] Y. Gao *et al.*, "Retrieval-augmented generation for large language models: A survey," *arXiv:2312.10997*, 2023.

Authorized licensed use limited to: Alliance University. Downloaded on October 14,2026 at 09:10:06 UTC from IEEE Xplore. Restrictions apply.